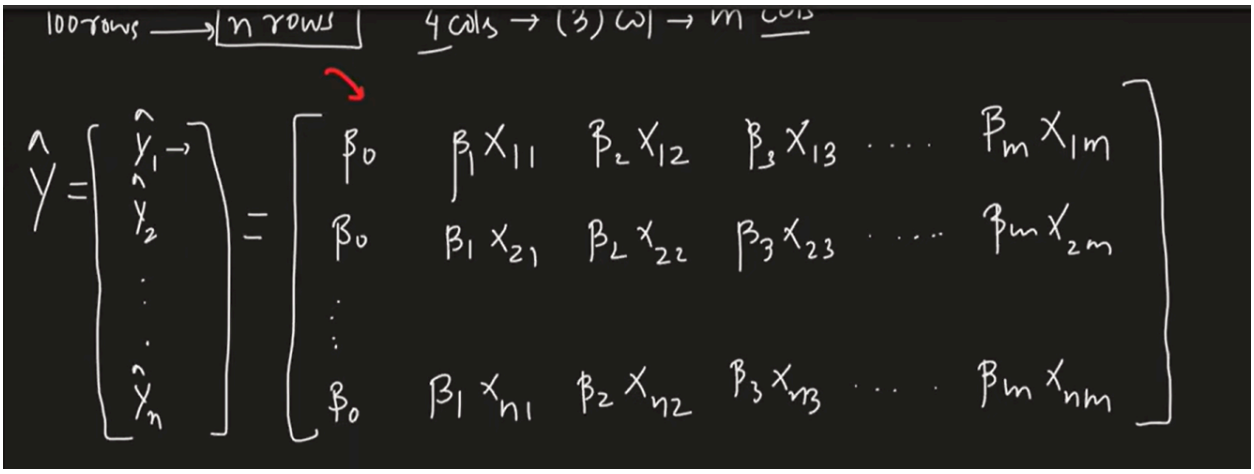
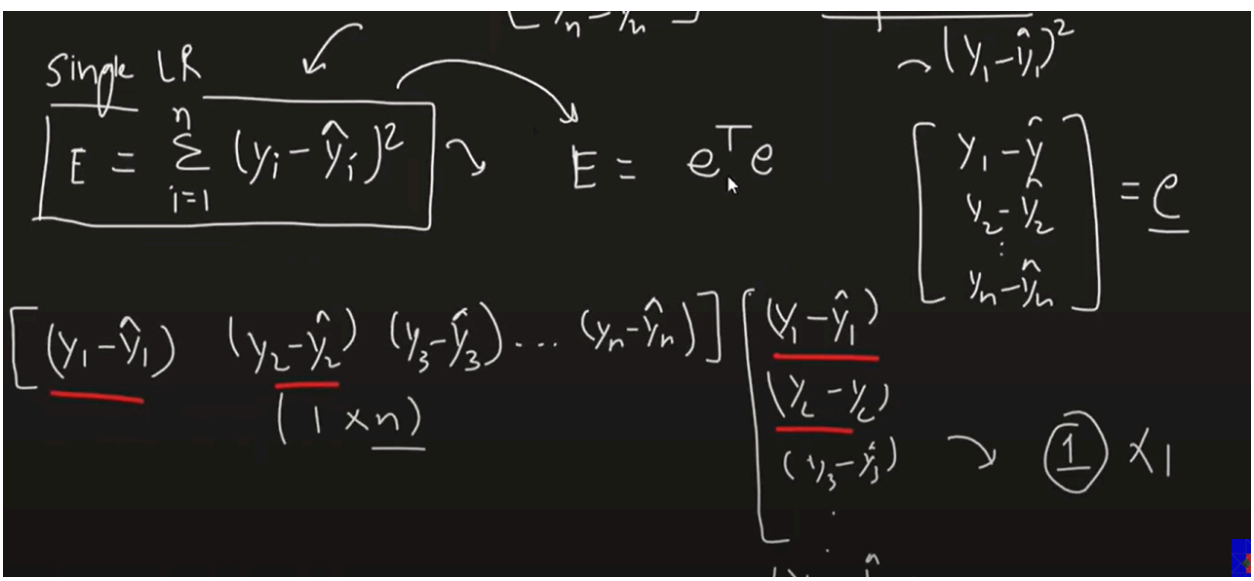
# Multilinear Regression

Here, n is the number of test cases and m is the input dimensions




From SLR  we know the error function E formula , we can put the shits in a matrix



$$(y_1-\hat{y}_1)^2+(y_2-\hat{y}_2)^2+(y_3-\hat{y}_3)^2+\dots+(y_n-\hat{y}_n)^2$$

$(n\times 1)$

$$\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

$$E=e^Te=(y-\hat{y})^T(y-\hat{y})$$

$$E=e^Te=(y-\hat{y})^T(y-\hat{y})$$

$$=(y^T-\hat{y}^T)(y-\hat{y})$$

$$=\left[y^T-(X\beta)^T\right](y-X\beta)$$

$$E=y^Ty-y^TX\beta-(X\beta)^Ty+(X\beta)^TX\beta$$

Detour

Saturday, May 15, 2021 4:56 PM

$(AB)^T = B^T A^T \quad y = A \quad X\beta = B \quad A^{T^T} = A$

$\boxed{y^T X\beta} = (X\beta)^T y \qquad (A^T B)^T = \underbrace{B^T A}_{RHS} \quad -(11)$

$A^T B = \underbrace{B^T A}_{RHS} \quad -(1)$

$A^T B = (A^T B)^T \longrightarrow A^T B = C$

$\boxed{C = C^T} \qquad A^T B \quad C = \boxed{y^T X\beta}$

$(y^T X\beta)^T = y^T X\beta$

$= \left[ y^T - (X\beta)^T \right] (y - X\beta)$

$E = y^T y - y^T X\beta - (X\beta)^T y + (X\beta)^T X\beta$

$E = y^T y - 2y^T X\beta + \beta^T X^T X\beta$

$\frac{dE}{d\beta} = \frac{d}{d\beta}\left[ y^T y - 2y^T X\beta + \beta^T X^T \right.$

$$E = y^T y - 2y^T X\beta + \beta^T X^T X \beta$$

matrix differ

$$\frac{dE}{d\beta} = \frac{d}{d\beta}\left[\underbrace{y^T y}_{\Downarrow 0} - 2y^T X\beta + \beta^T X^T X \beta\right] = 0$$

$$= 0 - 2y^T X + \frac{d}{d\beta}\left[\beta^T X^T X \beta\right] = 0$$

$$\frac{dy}{dA} = 2XA^T$$

$$y = A^T X A$$

$$= -2y^T X + 2X^T X \beta^T = 0$$

$$= \cancel{2} X^T X \beta^T = \cancel{2} y^T X$$

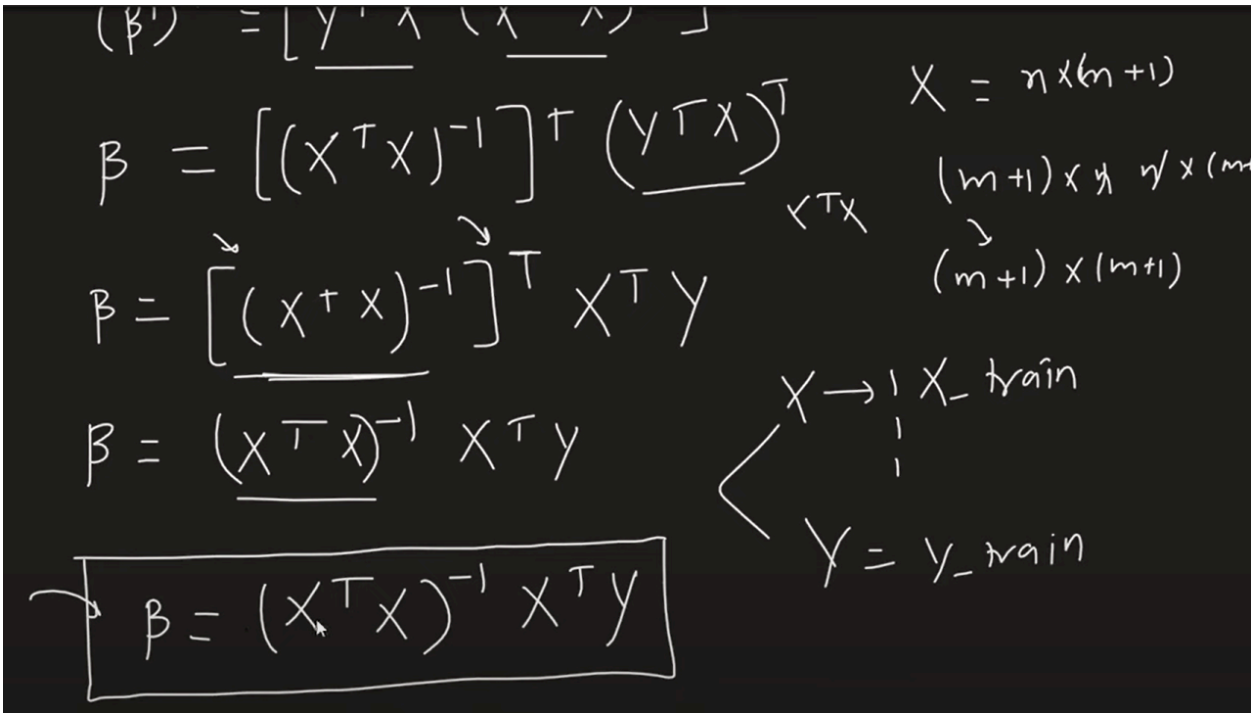
$$= -2y^T X + 2X^T X \beta^T = 0$$

$$= \cancel{2} X^T X \beta^T = \cancel{2} y^T X \frac{1}{(X^T X)}$$

$$\beta^T = y^T X (X^T X)^{-1}$$

$$(\beta^T)^T = \left[y^T X (X^T X)^{-1}\right]^T$$

$$\beta = \left[(X^T X)^{-1}\right]^T y^T X)$$




Raw Code :

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Load dataset
X, y = load_diabetes(return_X_y=True)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=2)

# Using Sklearn's Linear Regression
reg = LinearRegression()
```

```
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)

# Evaluate model
print("Sklearn R2 Score:", r2_score(y_test, y_pred))
print("Sklearn Coefficients:", reg.coef_)
print("Sklearn Intercept:", reg.intercept_)

# Custom Linear Regression Class
class MeraLR:
    def __init__(self):
        self.coef_ = None
        self.intercept_ = None

    def fit(self, X_train, y_train):
        X_train = np.insert(X_train, 0, 1, axis=1)  # Add bia
s term
        betas = np.linalg.inv(X_train.T @ X_train) @ X_train.
T @ y_train
        self.intercept_ = betas[0]
        self.coef_ = betas[1:]

    def predict(self, X_test):
        return X_test @ self.coef_ + self.intercept_

# Train and test custom model
lr = MeraLR()
lr.fit(X_train, y_train)
y_pred_custom = lr.predict(X_test)

# Evaluate custom model
print("Custom R2 Score:", r2_score(y_test, y_pred_custom))
print("Custom Coefficients:", lr.coef_)
print("Custom Intercept:", lr.intercept_)
```